📘 The ConnectX Backend Guide (For Future Tech Wizards)

Welcome! Imagine we are building a **Super Smart School** called **ConnectX**. This document will teach you exactly how this school works, who works there, and how they talk to each other.

🏗️ The Big Picture: Our School (Microservices Architecture)

Our backend isn't just one big giant robot; it's a team of smaller, specialized robots working together. This is called **Microservices**.

The Characters in Our School:

1. **Eureka Server (The Principal's Office / Phonebook)**
   - **What it does:** Every robot (service) has to report here when they start work. "Hi, I'm the Session Robot, and I'm at desk 8081!"
   - **Why?** If anyone needs to find the Session Robot, they check the Principal's Phonebook.
2. **Gateway (The School Gatekeeper)**
   - **What it does:** When students (Users/App) want to come in, they stop here first. The Gatekeeper checks where they want to go.
   - **Example:** "You want to create a session? Go to the Session Robot down the hall."
   - **Tech:** Spring Cloud Gateway.
3. **Session Service (The Session Manager)**
   - **What it does:** Creates special "rooms" of limited time (sessions) where two people can connect. It gives out tickets (Tokens).
   - **Tech:** Java, Spring Boot, Redis.
4. **Signaling Service (The Messenger)**
   - **What it does:** Helps two people send messages to each other to set up a video/file transfer. It checks with the Session Manager to make sure the "room" is valid.

🔍 Deep Dive: The Session Service (The Star of the Show)

Let's look inside the **Session Service**. This is where the magic happens for creating connections.

1. The Controller (`MyController.java`)

Analogy: The **Receptionist**.

- **What it does:** It sits at the front desk. When a request comes in (like "Create a Session"), the Receptionist takes the request and passes it to the Manager (Service). It doesn't do the work itself; it just directs traffic.
- **Key Methods:**
  - `create()` : "I need a new room!" -> Tells Service to make one.
  - `validate(id)` : "Is this ticket valid?" -> Asks Service to check.
  - `SenderConnected(id)` : "The sender is here!" -> Tells Service to note it down.

2. The Service (`SessionService.java`)

Analogy: The **Manager**.

- **What it does:** Does the actual thinking and logic.
- **Key Logic:**
  - **Creating a Session:**
    1. Generates a random ID (like "ABC-123").
    2. Calculates expiration times (Ticket expires in 5 minutes).
    3. Creates a "Session Card" (Entity).
    4. Saves it in the Filing Cabinet (Repository).
  - **Marking Connected:** When someone joins, update the card. If the Receiver joins, extend the time to 1 hour so they don't get kicked out!

3. The Repository (`SessionRepository.java`)

Analogy: The **Filing Cabinet**.

- **What it does:** Talk to the database. It puts things in and takes things out.
- **Special Skill:** It uses **Redis**.

4. The Entity (`SessionEntity.java`)

Analogy: The **Form/Card**.

- **What it does:** It's just a piece of paper that holds data: `sessionId`, `createdAt`, `expiresAt`, `senderConnected`. It represents one session.

🔍 Deep Dive: The Signaling Service (The Messenger)

This robot helps the Sender and Receiver talk to each other to set up their direct line (WebRTC). It doesn't see the video or files; it just passes the setup notes.

1. The Configuration (`WebSocketConfig.java`)

Analogy: The **Switchboard Setup**.

- **What it does:** It tells the school: "If anyone plugs a cable into the wall socket labeled `/ws`, send them to the Signaling Operator."
- **Code:** `registry.addHandler(handler, "/ws").setAllowedOrigins("*")` (Open to everyone).

2. The Handler (`SignalingWebSocketHandler.java`)

Analogy: The **Switchboard Operator**.

- **What it does:** This is the brain of the Signaling Service. It manages the live phone calls (WebSocket connections).
- **Key Duties:**
  - **Validation (Security Guard):** When you try to connect, it checks your `token`.
    - "Let me see your ID." (Checks JWT).
    - "Let me call the Session Manager to see if this room actually exists." (Calls `SessionServiceClient`).
  - **Connecting (Routing):**

        - It keeps two lists: a list of **Senders** (`senderMap`) and a list of **Receivers** (`receiverMap`).
    - **Passing Notes (Forwarding):**
        - If Sender says "Here is my OFFER" (WebRTC technical term), the Operator looks up the Receiver for that session and hands them the note.
        - If Receiver says "Here is my ANSWER", the Operator hands it to the Sender.
        - It handles types like `JOIN_AS_SENDER` , `OFFER` , `ANSWER` , `ICE_CANDIDATE` , `FILE_METADATA` .

3. The Client (`SessionServiceClient.java`)

Analogy: The **Intercom to the Manager**.

- **What it does:** It allows the Signaling Service to talk to the Session Service over HTTP.
- **Why?** The Signaling Service doesn't have the database of sessions. Only the Session Service does. So it has to ask: "Hey, is Session ID 'ABC-123' valid?"

📜 The Rule Book: `application.yml` Explained

This file is like the **Instruction Manual** for the robot. It tells the robot how to behave when it wakes up.

```
spring:
  application:
    name: session-service  # "My name is Session-Service"
server:
  port: 8081               # "I sit at Desk #8081"
  address: 0.0.0.0         # "I listen to anyone who talks to me"

eureka:
  client:
    service-url:
      defaultZone: http://192.168.29.245:8761/eureka # "The Principal is at this address. I must call him!"
    register-with-eureka: true  # "Yes, please add my name to the phonebook."
    fetch-registry: true        # "Please give me the phonebook so I can find others."
```

🛠️ Why This Tech? (The "Why Not That?" Section)

1. **Redis** (The Database)

- **What we use:** Redis (In-Memory Database).
- **Why?** Redis is like a **Whiteboard**. Writing on it is SUPER fast, but if you turn off the power, it gets erased (mostly).
- **Why not SQL (MySQL/Postgres)?** SQL is like a **Stone Tablet**. It keeps things forever and is structured, but it's heavier.
    - **Reason:** Our sessions only last 5 minutes to 1 hour. We don't need to keep them forever. Redis handles "Self-Destructing" data (TTL) automatically. SQL would require us to write a cleanup script to delete old rows constantly.

2. **Spring Boot** (The Framework)

- **What we use:** Spring Boot.
- **Why?** It's like a **Magic Backpack** that has everything pre-packed (Server, Security, Configs). You just open it and start working.
- **Why not pure Java?** You'd have to build the server, the routing, and the database connections from scratch. Too much work!

3. **Eureka** (Service Discovery)

- **What we use:** Eureka.
- **Why?** Dynamic finding. If we start 10 Session Robots, Eureka knows about all 5.
- **Alternative:** Hardcoding IP addresses (e.g., `localhost:8081`).
  - **Problem:** If the robot moves to a different computer, the hardcoded address breaks. Eureka automates this.

🗣️ How Services Talk (Communication)

1. **User -> Gateway**: User knocks on the door (Port 8080).
2. **Gateway -> SessionService**: Gateway looks at the address (`/sessions`) and sends it to SessionService.
3. **SignalingService -> SessionService**:
   - **Currently:** The Messenger (Signaling) uses a **Phone** (`WebClient`) to call the Session Manager directly.
   - **Improvement:** It currently calls `localhost:8081`. It *should* call `http://session-service` so it uses the Phonebook (Eureka) to find it dynamically!

🎓 Top 10 Interview Questions

1. **Q: What is the difference between `@Controller` and `@RestController` in Spring Boot?**
   - **A:** `@Controller` returns Views (HTML pages), while `@RestController` returns Data (JSON/XML). We use `@RestController` for our backend APIs.
2. **Q: Why did you use Redis for this project instead of MySQL?**
   - **A:** Because sessions are temporary (ephemeral). Redis supports TTL (Time-To-Live), which auto-deletes data after a set time, making it perfect for short-lived sessions without manual cleanup.
3. **Q: What is Eureka and why do we need it?**
   - **A:** Eureka is a Service Registry. In a microservices architecture, services might change IP addresses. Eureka acts as a dynamic phonebook so services can find each other without hardcoded IPs.
4. **Q: Explain the lifecycle of a Bean in Spring.**
   - **A:** Instantiate -> Populate Properties -> `setBeanName` -> `setBeanFactory` -> Pre-Initialization (PostProcessors) -> `afterPropertiesSet` -> Custom Init -> Post-Initialization -> Ready -> Destroy.
5. **Q: What is the purpose of the `application.yml` file?**
   - **A:** It centralizes configuration properties (Database URLs, Server Ports, Service Names) so we can change settings without rewriting Java code.
6. **Q: How does the Gateway facilitate Rate Limiting?**

- **A:** Using Spring Cloud Gateway filters (like the generic RequestRateLimiter we saw in the config). It uses Redis (Token Bucket algorithm) to count requests and reject them if they exceed the limit (e.g., 5 requests/sec).

7. **Q: What is Dependency Injection (DI)?**
   - **A:** It's a design pattern where Spring "injects" the objects your class needs (like injecting `SessionRepository` into `SessionService`) instead of the class creating them itself using the `new` keyword.
8. **Q: What is the difference between `@Service`, `@Repository`, and `@Component`?**
   - **A:** Technically they are the same (Stereotypes), but:
     - `@Repository` handles Data Access (and DB exceptions).
     - `@Service` handles Business Logic.
     - `@Component` is a generic bean.
9. **Q: How does `WebClient` differ from `RestTemplate`?**
   - **A:** `RestTemplate` is blocking (synchronous). `WebClient` is non-blocking (asynchronous/reactive), meaning it can handle more concurrent requests without freezing threads.
10. **Q: What is the underlying data structure used for Session Storage in your code?**
    - **A:** A Redis **Hash**. We use `opsForValue().set()` (actually String/Value) in the custom repository code shown, or `@RedisHash` if using the repository interface. *Correction based on code: The Repository uses `opsForValue().set()`, which actually stores it as a String/Value (likely serialized JSON or Binary), not a Redis Hash structure, despite the annotation on the Entity.*
11. **Q: How does WebSocket communication differ from HTTP?**
    - **A:** HTTP is "Request-Response" (Client asks, Server answers, then hangs up). WebSocket is "Full-Duplex" (Open phone line). Client and Server can talk to each other freely at any time without hanging up.
12. **Q: What is the role of the `ConcurrentHashMap` in the WebSocket Handler?**
    - **A:** It stores the active WebSocket sessions (`sessionId -> WebSocketSession`). We use `ConcurrentHashMap` because multiple users ("Threads") are connecting/disconnecting at the same time, and this map is thread-safe (preventing crashes when two people update the list at once).
13. **Q: What happens if the `SessionService` goes down while users are connected to `SignalingService`?**
    - **A:** Existing WebSocket connections might stay alive (since they are in memory in SignalingService), but *new* users won't be able to join because `sessionClient.isValidSession()` will fail when it tries to check with the dead SessionService.
14. **Q: Explain "Signaling" in the context of WebRTC.**
    - **A:** WebRTC allows Peer-to-Peer (P2P) connection, but the two peers don't know each other's IP address or capabilities initially. "Signaling" is the process of exchanging this setup information (SDP Offer/Answer, ICE Candidates) via a server (our SignalingService) before the direct P2P connection is established.
15. **Q: Why do we use `TextWebSocketHandler` instead of `BinaryWebSocketHandler`?**

- **A:** We are exchanging JSON text messages (Offers, Answers, Metadata). We are NOT sending the actual File bytes through the WebSocket (that happens over the direct WebRTC connection). So, text mode is easier and Human-readable.